Lab 2 - Instructions

Basics of numpy and matplotlib

Software 1 - Mathematics

Information and Communication Technology

Metropolia University of Applied Sciences

What you'll learn

In this laboratory exercise, you'll develop essential skills for visualizing and solving mathematical problems graphically. Instead of solving equations with pure algebra, you'll use graphs as visual tools to find solutions.

By the end of this lab, you will be able to:

- **Visualize mathematical functions** - Create smooth, accurate plots of functions using NumPy and Matplotlib
- **Find roots graphically** - Discover where functions equal zero by zooming into graphs
- **Find intersection points** - Determine where two functions meet by plotting them together
- **Master the zoom technique** - Use `xlim()` and `ylim()` to create detailed views of specific regions
- **Communicate results clearly** - Decorate graphs with titles, labels, legends, and grids

The "Zoom In" strategy

The key technique you'll master is the **iterative zoom strategy**:

1. **Overview** → Create a wide view to see the overall function behavior
2. **Identify** → Locate regions of interest (roots, intersections)
3. **Zoom** → Create separate, detailed graphs focusing on each region
4. **Read** → Extract numerical values from the zoomed graphs

Think of this like Google Maps: start with a city view, identify your destination, then zoom to street level to see the exact address.

Expected solutions

- **Problem 1:** Find 4 roots graphically (create 4 separate zoomed graphs)
- **Problem 2:** Find 2 intersection points graphically (create 2 separate zoomed graphs)

Before you begin

Setup checklist

1. Complete the Moodle Quiz Lab 2
2. Download `labx.py` to the same folder as the notebook
3. Ensure `labx.py` and your notebook are in the same project folder
4. Change your `student_id` in the Setup section in the notebook

Common setup error

If you see `NameError: name 'mx' is not defined`, the `labx.py` file is not in the correct folder.

Setup - Important notes

Import libraries ONCE

After running the setup cell, the libraries are loaded into your notebook's memory. You do NOT need to repeat these import statements later in your notebook.

```
from IPython.display import Markdown, display
import numpy as np
import matplotlib.pyplot as plt
import labx as mx
```

Problem 1 - Finding the roots

Create graphical plot of the function `mx.lab_2_problem_1(x, student_id)`.

Step 1: Create an overview graph

- Plot the values in range x ∈ [-10, 10] (note: negative to positive)
- Use proper step size to make the plot smooth. Recommended:
  - `x = np.linspace(-10, 10, 1000)` or
  - `x = np.arange(-10, 10, 0.01)`
- **Important:** Do NOT use `np.arange(-10, 10)` or `np.arange(-10, 10, 1)` - this creates too few points!

Step 2: Find ALL roots graphically

- A root is where the function crosses the x-axis (where f(x) = 0, meaning y = 0)
- There are **4 roots** for this function - make sure you find all of them!
- Look at your overview graph and identify approximately where each root is located

Step 3: Create separate zoomed graphs

- Create **4 separate graphs** (one for each root)
- Use `plt.xlim()` and `plt.ylim()` to zoom in to each root location
- Zoom in enough so you can read the x-value with **two decimal accuracy**
- Make sure your zoomed graph shows where the function crosses y = 0 (not x = 0!)

Example structure

```
# First overview graph
plt.figure()
plt.plot(x, y)
plt.title("Overview: All roots visible")
plt.grid(True)
plt.show()

# Then separate graphs for each root
plt.figure()
plt.plot(x, y)
plt.xlim(-8, -6)  # Example: adjust to zoom to first root
plt.ylim(-5, 5)   # Adjust to show y=0 line clearly
plt.title("Root 1 (zoomed)")
plt.grid(True)
plt.show()
```

```
# Repeat for roots 2, 3, and 4...
```

Remember to decorate ALL plots

- Title (use `plt.title()`)
- X and Y axis labels (use `plt.xlabel()` and `plt.ylabel()`)
- Grid lines (use `plt.grid(True)`)

Problem 2 - Finding the intersection points

CRITICAL FIRST STEP

Before you start coding, you MUST run the following code cell to see YOUR personalized functions:

```
txt = mx.lab_2_problem_2(student_id)
display(Markdown(txt))
```

Common mistake

Many students skip this step and use wrong functions! Each student gets different functions based on their student_id.

Step 1: Create an overview graph

- Plot BOTH functions (f(x) and g(x)) on the same graph
- Use the x range shown in your personalized instructions (typically $x \in [0, 10]$)
- Use enough points: `x = np.linspace(start, end, 500)` or similar
- Add a legend to distinguish between the two functions: `plt.legend(['f(x)', 'g(x)'])`

Step 2: Identify intersection points

- Look at your overview graph
- There are typically **2 intersection points**
- identify approximately where they are
- An intersection point is where the two functions have the same value (where the lines cross)

Step 3: Create separate zoomed graphs

- Create **2 separate graphs** (one for each intersection point)
- For each graph
  - Plot BOTH functions
  - Use `plt.xlim()` and `plt.ylim()` to zoom in to the intersection
  - Make the zoom tight enough to read the x-coordinate with good accuracy
  - Include the legend

Example structure

```
# Overview graph
plt.figure()
plt.plot(x, f, label='f(x)')
plt.plot(x, g, label='g(x)')
plt.title("Overview: Both functions")
plt.legend()
plt.grid(True)
plt.show()

# Zoomed graph for first intersection
plt.figure()
plt.plot(x, f, label='f(x)')
plt.plot(x, g, label='g(x)')
plt.xlim(1, 3)  # Example: adjust to zoom to first intersection
plt.ylim(0, 5)  # Adjust accordingly
plt.title("Intersection Point 1 (zoomed)")
plt.legend()
plt.grid(True)
plt.show()

# Repeat for second intersection point...
```

Remember

Decorate all plots with title, axis labels, legend, and grid lines.

Common errors and solutions

Error: `NameError: name 'mx' is not defined`

Problem

- labx.py file is not downloaded from OMA or it locates in different folder than your notebook

Solution

- Make sure the `labx.py` file is in the same folder as your notebook
- Check that both files are in your PyCharm project folder

Error: Rough/jagged graphs

Problem

- Not using enough points, for example
  - x = `np.arange(-10, 10)` or x = `np.arange(-10, 10, 1)`

Solution

- Use `np.linspace(-10, 10, 1000)` or `np.arange(-10, 10, 0.01)`

Can't find all roots/intersections

Check

- Did you change the `student_id` to YOUR student number?

Check

- Did you run ALL cells from the beginning? (Menu: Run → Run All Cells)

How to Succeed

DO

- ✅ Run ALL code cells in sequence from the beginning
- ✅ Execute the instruction display cells (especially `mx.lab_2_problem_2(student_id)`)
- ✅ Create **separate graphs** for each root and intersection point
- ✅ Use enough points: `np.linspace(-10, 10, 1000)` - not `np.arange(-10, 10)`
- ✅ Zoom in enough to read values with **2 decimal accuracy**
- ✅ Decorate all graphs with titles, labels, grids, and legends

DON'T

- ❌ Skip running the display instruction cells
- ❌ Show multiple roots/intersections in the same zoomed graph
- ❌ Use too few points (rough graphs mean insufficient detail)
- ❌ Zoom to x=0 when you need y=0 (for roots)
- ❌ Repeat import statements throughout your notebook